Qianqian Lin, Shuling Wang, Bohua Zhan, Bin Gu. Modelling and Verification of the RTPS Protocol Using UPPAAL and Simulink/Stateflow. JOURNAL OF COMPUTER SCIENCE AND TECHNOLOGY 33(1): 1–27 July 2020. DOI 10.1007/s11390-015-0000-0

Modelling and Verification of the Real-Time Publish and Subscribe Protocol using UPPAAL and Simulink/Stateflow

Qianqian Lin[1], Shuling Wang[1,*], Member, CCF, Bohua Zhan[1,*], Member, CCF, and Bin Gu[2,*]

, Member, CCF
[1] *State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences, Beijing 100190, China*
[2] *Beijing Institute of Control Engineering, Beijing 100081, China*

E-mail: linqq@ios.ac.cn; wangsl@ios.ac.cn; bzhan@ios.ac.cn; gubinbj@sina.com

Received April 11, 2020; revised September 10, 2020.

Abstract Real-Time Publish and Subscribe (RTPS) protocol is a protocol for implementing message exchange over an unreliable transport in the Data Distribution Service (DDS). Formal modelling and verification of the protocol provide stronger guarantees of its correctness and efficiency than testing alone. In this paper, we build formal models for the RTPS protocol using Uppaal and Simulink/Stateflow. Modelling using Simulink/Stateflow allows analyzing the protocol through simulation, as well as generating executable code. Modelling using Uppaal allows us to verify properties of the model stated in TCTL (Timed Computation Tree Logic), as well as estimating its performance using statistical model checking. We further describe a procedure for translation from Stateflow to timed automata, where a subset of major features in Stateflow is supported, and prove the soundness statement that the Stateflow model is a refinement of the translated timed automata model. As a consequence, any property in a certain fragment of TCTL that we have verified for the timed automata model in Uppaal is preserved for the original Stateflow model.

Keywords Real-Time Publish and Subscribe (RTPS), modelling, verification, UPPAAL, Simulink/Stateflow

1 Introduction

Data Distribution Service (DDS) [1] is a portable middleware technology which provides an interface of information transmission among applications and operating systems. Adopting the Publish/Subscribe architecture, DDS takes data as the center, and provides rich QoS (quality of service) strategy to realize data management and data sharing in the whole network [2]. It is widely used in aerospace and national defense fields requiring high performance, predictability and effective use of resources. The Real-Time Publish and Subscribe (RTPS) protocol [3, 4] has been standardized by the OMG (Object Management Group) as an interoperability protocol implemented using DDS. The protocol specifies requirements that any implementation must satisfy in order to be interoperable with each other. These include the structure of entities involved in the conversation network, format for messages and how they are exchanged, and a mechanism for discovering other participants. The protocol allows flexibility in the implementation, including the choice of best-effort or reliable communication, and how much state to keep at the endpoints.

Regular Paper

This work was partially supported by the National Natural Science Foundation of China under Grant Nos. 61625206, 61972385 and 61732001, and by Chinese Academy of Sciences Pioneer Hundred Talents Program under grant No. Y9RC585036.

*Corresponding Author

In this paper, we build formal models for the RTPS protocol using Simulink/Stateflow and UPPAAL. The models describe detailed behaviors specified by the protocol, including its real-time and probabilistic aspects. To avoid making the model overly complex, some aspects of the protocol are not considered, as described in detail in the body of the paper. Such modelling allows the protocol to be analyzed by simulation, as well as rigorous verification using model checking. It also allows automatic code generation to implementations of the protocol. This provides stronger guarantees for the correctness and efficiency of the protocol than testing alone.

Simulink/Stateflow[1] is a de-facto standard in modelling, analysis, and design of embedded systems. It provides a strong numerical simulation engine. Meanwhile, the built-in Simulink Coder supports C/C++ code generation from Simulink/Stateflow models. However, due to the incomplete coverage of numerical simulation, the correctness of the models cannot be fully guaranteed. UPPAAL [5, 6] is an integrated tool environment for modelling, validation and verification of real-time systems modelled as networks of timed automata (TA). It allows model checking of specifications written in Timed Computation Tree Logic (TCTL), as well as statistical model checking of specifications in Metric Interval Temporal Logic (MITL).

We will take full advantage of both Simulink/Stateflow and UPPAAL. First, we model the RTPS protocol as Stateflow charts and analyze its behavior using simulation. Second, we provide a translation from Stateflow models to UPPAAL timed automata, and prove formally that the original Stateflow model is a refinement of the translated TA model. Thus, any property in a certain fragment of TCTL that holds for the TA model must also hold for the original Stateflow model. Third, we verify using UPPAAL the correctness of the TA model according to the RTPS specification. As a consequence, the original Stateflow model is also verified. We also estimate the performance of the TA model using statistical model checking. Finally, we generate C code from the Stateflow model using Simulink Coder. The execution of the generated code agrees with the simulation and the description of the protocol.

Using the above approach, we verify several important properties of the RTPS protocol. They include: the writers and readers send and receive data in a consistent and sequential way; and the heartbeat and acknowledgement mechanisms work correctly to ensure reliable communication. Moreover, using statistical model checking, we determine the quantitative properties of the protocol. In particular, we estimate the probability of finishing the transmission of some fixed input data within a certain time limit, as a function of the communication failure rate, as well as various delay times in the model. This shows the potential of the formal model to provide guidance on the tuning of parameters when deploying an implementation.

After presenting the related work in Section 2, the rest of the paper is organized as follows. In Section 3 we review some preliminaries, including the RTPS protocol, Simulink/Stateflow, and UPPAAL. In Section 4 we describe the model of the RTPS protocol as Stateflow charts. In Section 5, we present the procedure to translate Stateflow charts to timed automata and formally prove the soundness of the translation. In Section 6 we verify various properties of the protocol using (statistical) model checking. Finally, we conclude the paper in Section 7 with a discussion of future work.

[1] https://ww2.mathworks.cn/products/simulink.html.

2 Related Work

Analysis and verification of many aspects of DDS have been explored in [7, 8, 9], including real-time performance, security of DDS-based middleware, and so on. Alaerjan et al. [10] defined the missing functional behavior in the DDS dynamic model and the semantics of the new behavior using Object Constraint Language (OCL) [11]. The work by Liu et al. [12] mainly focused on DDS in ROS2 to verify its security, activity and priority. Yin et al. [13] modelled and verified the communication dependability of the reliable StatefulWriter module in RTPS using Communicating Sequential Processes (CSP) [14, 15] and the model checker Process Analysis Toolkit (PAT). It is based on a description of data transmission to a single reader without data loss. In contrast, our work considers the reliability of transmission when data loss is allowed, as well as sending data to multiple readers. We also model the history cache in the writer and the reader separately. Finally, our approach supports the automatic generation of C code, and through statistical model checking, the determination of the influence of timing parameters on the model performance.

Translations from Stateflow (subsets) to Uppaal have been addressed previously [16, 17]. Compared with our work, they consider more advanced features of Stateflow, such as supertransition, event stack and execution interruption mechanism. The resulting TAs are quite complex, making them expensive to verify in Uppaal. For example, each Stateflow state is translated to four cooperative parallel automata in [16]. Moreover, no formal guarantee is provided for the correctness of the translation. As a result, whether the properties proved for the TA is preserved for the Stateflow model is not ensured. In this paper, we consider a more limited subset of Stateflow that is sufficient for modelling and analyzing real-time systems such as the RTPS protocol. This results in simpler translated TA models that can be verified easily in Uppaal. We further prove formally the correctness of the translation based on the simulation of transition semantics. The semantics of Stateflow or its translation to other formal languages have also been studied in [18, 19, 20, 21].

3 Preliminaries

3.1 RTPS Protocol

The RTPS (Real-Time Publish and Subscribe) protocol specifies a mechanism for transmitting data between endpoints on top of the DDS wire protocol. Its real-time aspect comes from allowing delays at various stages of data transmission, with tunable constraints on those delays. It also has a probabilistic aspect as it allows unreliable communication channels, with a mechanism for maintaining reliable data transmission over such channels. We will take all these behaviors into consideration in the modelling and verification.

The protocol consists of two parts: the Platform Independent Model (PIM) and the Platform Specific Models. The PIM specifies requirements that are unrelated to the platform used for transmitting data. It consists of four modules describing different aspects of the protocol. The structure module describes the structure of RTPS entities and the relationships among them. The entities include domain, participants, endpoints (readers and writers), and the history cache. The message module describes the format of messages and their logical interpretation. The behavior module specifies valid sequences of message exchanges between entities. This is the part that we will mainly focus on in this paper. The discovery module specifies how RTPS endpoints are configured and paired with each other.

The behavior module includes several reference implementations, allowing choices along two main dimen-

sions: whether the communication is best-effort or reliable, and what states the readers and writers maintain about the available messages. We focus on the most complex (and interesting) combination: reliable communication while maintaining state about all available messages. Following the protocol document, we call the modelled entities Reliable StatefulWriter and Reliable StatefulReader.

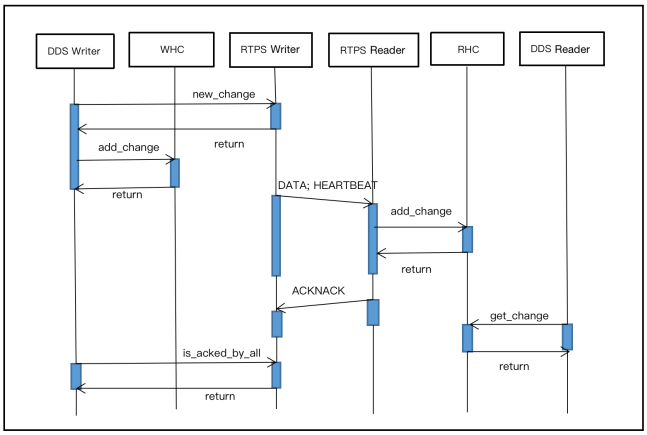

Fig.1. High-level overview of the message exchange according to the RTPS protocol.

Fig. 1 gives a high-level overview of how messages are exchanged according to the RTPS protocol. Each DDS Writer and each DDS Reader are paired with an RTPS Writer and RTPS Reader, respectively. Each RTPS Writer and each RTPS Reader contain a HistoryCache, which maintains a number of CacheChange objects. The RTPS Writer and the RTPS Reader also maintain state recording the status of each available message (sent, received, acknowledged, etc.). The aim is to transfer data from the DDS Writer (publisher) to the DDS Reader (subscriber). The transfer consists of several steps.

1. For each new message, the DDS Writer adds the message as a CacheChange to the HistoryCache of its associated RTPS Writer.

2. When there are unsent CacheChanges in the HistoryCache, the RTPS Writer attempts to send them to each of the matched readers. Note the messages may be lost as we do not assume the communication channel is completely reliable.

3. The RTPS Writer expects each reader to acknowledge receipt of the messages. As long as there are messages that are unacknowledged, it sends heartbeat messages at regular intervals to each of the readers.

4. The RTPS Reader, on receiving a data message, updates its HistoryCache. On receiving a heartbeat message, it may respond by acknowledging receipt of messages or signal that some of the messages are not received, using an acknack message.

5. The RTPS Writer, on receiving a response from the reader signalling that some messages are not received, re-sends the relevant messages. This continues until all messages are acknowledged.

6. Finally, the DDS Reader requests messages from its associated RTPS Reader, reading from its HistoryCache.

The RTPS protocol specifies a period for heartbeats and allows delayed responses to heartbeats and acknowledgements. In the implementation, the end-user can tune these timing characteristics according to application requirements, deployment configuration and underlying transports. Some of the tunable parameters are listed in Table 1.

The above behaviors will be modelled in Section 4, including the timing properties and the possible failures of the communication channel. On the other hand, to avoid making the model overly complex, the following aspects of the RTPS protocol are not considered in this paper: sending data by dividing into several fragments,

Table 1. Tuning parameters in the RTPS protocol

Parameter	Scope	Meaning
heartbeatPeriod	RTPS Writer	The period for the RTPS Writer to repeatedly announce the availability of data by sending a heartbeat message.
nackResponseDelay	RTPS Writer	The delay for the RTPS Writer to respond to a request for data from a negative acknowledgment.
heartbeatResponseDelay	RTPS Reader	The delay for the RTPS Reader to send a positive or negative acknowledgment.

and heartbeat/acknowledgement protocol for data fragments; relaying of messages, and the associated protocol for specifying source, destination, and reply addresses in a message; protocol for discovering new participants, as specified in the discovery module; behavior specific to the platform used, as given in Platform Specific Models.

3.2 Simulink/Stateflow

Simulink is an environment for model-based development of embedded systems. It provides a rich set of fixed-step and variable-step solvers for simulating system models. Moreover, Simulink supports the generation of executable C/C++ code using the built-in Simulink Coder. Therefore, after a Simulink model is built, we can analyze its behavior by simulation first, and then use Simulink Coder to produce C/C++ code, whose execution should have the same behavior as simulations of the model.

Stateflow is a toolbox of Simulink, adding facilities for modelling and simulating reactive systems in the form of statecharts. It can be defined as Simulink blocks, fed with Simulink inputs and producing Simulink outputs. A Stateflow chart is composed of transitions, states and junctions. Each transition is labelled with $E[C]\{cAct\}/tAct$, where E is an event, C is the condition, $cAct$ the condition action, and $tAct$ the transition action. The event E triggers the transition to take place, provided that the condition C is true, the action $cAct$ will be executed immediately, while $tAct$ will be executed when a valid transition path reaching a target state is completed. A state is labelled by three optional types of actions: entry action, during action, and exit action. Directed event broadcasting is supported by Stateflow, which has the form $\text{send}(e, S)$, where event e is broadcast to state S and its substates. Stateflow supports construction of flow charts using connective junctions and transitions, which can be used between states to specify decision logics in transition networks.

Stateflow charts are organized as hierarchical diagrams: Or diagram, for which the states are mutually exclusive and only one state becomes active at a time, and And diagram, for which the states are parallel and all of them become active simultaneously. Although Stateflow allows multiple transitions originating from a state, as well as multiple parallel states in an And diagram, its behavior is deterministic, by assigning different priorities to transitions and states. The Stateflow charts are activated by triggering events from the outside Simulink diagram, or at specified rates corresponding to the sample times. We make use of the second case in the modelling in this paper.

3.3 UPPAAL

Uppaal is a toolbox for modelling and analysis of real-time systems, modelled as networks of timed automata (TA) [22].

Formally, a timed automaton is a tuple (L, l_0, C, A, E, I), where L is a set of locations, $l_0 \in L$ the initial location, C the set of clocks, A the set of actions, co-actions and the internal τ-action, $E \subseteq L \times A \times B(C) \times 2^C \times L$ the set of edges between

locations with an action, a guard and a set of clocks to be reset, and $I : L \to B(C)$ assigns invariants to locations. The semantics can be defined as a labelled transition system $\langle S, s_0, \to \rangle$, where $S \subseteq L \times \mathbb{R}^C$ is the set of states, $s_0 = (l_0, u_0)$ is the initial state, and $\to \subseteq S \times (\mathbb{R}_{\geq 0} \cup A) \times S$ is the transition relation such that

- $(l, u) \xrightarrow{d} (l, u + d)$ if $\forall d' : 0 \leq d' \leq d \Rightarrow u + d' \in I(l)$;
- $(l, u) \xrightarrow{a} (l', u')$ if there exists $e = (l, a, g, r, l') \in E$ such that $u \in g, u' = [r \mapsto 0]u$ and $u' \in I(l')$,

which correspond to time progress and action execution, respectively.

In UPPAAL, the TAs synchronise with each other using channels which carry no data values. Asynchronous communication is provided by broadcast channels. The sender of a broadcast channel is never blocked, and the message can be received by 0, 1, or multiple receivers. Committed and urgent locations are supported, which must be exited without passage of time. Finally, more general predicates involving shared variables are permitted as guards of transitions.

UPPAAL uses a simplified version of TCTL [23] to express requirement specifications. It also supports statistical model checking (SMC) [9, 24] in the UPPAAL SMC extension. UPPAAL SMC avoids the construction of the full state space, thus avoiding the state explosion problem. It checks properties by performing a certain number of simulation runs and then uses statistical techniques to estimate the probability that each property is satisfied. The use of SMC introduces certain restrictions on the timed automata, for example allowing only broadcast synchronisations. The language used to express properties to be checked by SMC is an extension of metric interval temporal logic (MITL) [9, 25]. General queries are in the form `Pr[(clock|step_num|void) <= bound]`ψ, representing the probability that ψ is satisfied before the clock or step number reaches the given bound.

4 Modelling and Simulation Using Stateflow

In this section, we describe the modelling and simulation of the RTPS protocol using Stateflow. We consider the case with one RTPS Writer and two matched RTPS Readers. The model consists of three kinds of charts: *Channel* models a communication channel with possible message loss; *RTPS_Writer* models a Reliable StatefulWriter; and *RTPS_Reader* models a Reliable StatefulReader.

4.1 Messages

There are three kinds of messages in our model: MSG_{data}, $MSG_{\text{heartbeat}}$ and MSG_{acknack}. For each kind of messages, we define in Stateflow a multi-dimensional array storing the transmitted messages. The arrays corresponding to the three kinds of messages are *buffer_data*, *buffer_heartbeat*, and *buffer_acknack*, respectively. When a message needs to be sent, the Stateflow model triggers a corresponding event, and at the same time modifies the buffer associated to that kind of message.

Data message MSG_{data} represents data-change sent from writers to readers, as well as gap information to notify that certain sequence numbers of data are no longer relevant. The structure of MSG_{data} is as follows:

$$MSG_{\text{data}} = (WriterId, ReaderId, WriterSN, Data)$$

The fields *WriterId* and *ReaderId* identify the sender and the receiver of the data-change respectively. *WriterSN* is a sequence number that uniquely identifies the change. *Data* represents the data to be transmitted.

We choose to represent data and gap messages uniformly as MSG_{data}, with gap messages indicated by

setting -1 to field *Data*, and data messages assumed to be non-negative numbers. This is a simplification compared with the original protocol, where gaps form a separate kind of messages and may indicate a whole range of sequence numbers to be no longer relevant.

Heartbeat message The heartbeat and liveliness mechanisms are implemented using $MSG_{\text{heartbeat}}$ messages. The structure of $MSG_{\text{heartbeat}}$ is as follows:

$$MSG_{\text{heartbeat}} = (WriterId, ReaderId, FinalFlag, LivelinessFlag, FirstSN, LastSN)$$

The field *FinalFlag* indicates whether the reader is required to respond to the heartbeat message. The field *LivelinessFlag* indicates whether the DDS DataWriter associated with the RTPS Writer has manually asserted its liveliness. The *FirstSN* and *LastSN* fields identify the first (lowest) sequence number and the last (highest) sequence number that are available in the writer. The functions *set_Final_Flag*(), *set_Liveliness_Flag*() and *set_heartbeatSN*() are used to update information according to the current status of the RTPS Writer while the function *get_heartbeat*() reads the relevant information from the array *buffer_heartbeat* on behalf of the RTPS Reader.

Acknack message To implement the acknowledgement mechanism, MSG_{acknack} is used to communicate the state of an RTPS Reader to an RTPS Writer, informing both positive and negative acknowledgments. The structure of the message is as follows:

$$MSG_{\text{acknack}} = (WriterId, ReaderId, AckNum, NackNum)$$

AckNum is the last sequence number the reader has received continuously and *NackNum* is the last sequence number that is available (as indicated through heartbeat messages) but still missing in the reader. Here we make some simplifications to the original acknack mechanism, where the set of missing sequence numbers is described by a sequence of bits. On the RTPS Writer side, every time the writer receives the message, *get_acknack*() is used to get information including *AckNum* and *NackNum* from the message. On the RTPS Reader side, the message is built using the function *set_acknack*().

4.2 Channels

The Stateflow chart *Channel* models the possibility of losing a message in the communication channel, as well as a time delay when (successfully) sending a message. Fig. 2 shows the chart for the case of sending data messages from the writer to two readers. The locations *try* and *next* represent the waiting state and the state after attempting to send to the first reader, respectively. On receiving the event TRY indicating an attempt to send a message, the function *channel*() is used to determine whether the message is lost. It returns 1 with probability *PassingRate*, and 0 otherwise. If the message is not lost, the SUCCESS event is sent to the corresponding RTPS Reader after delaying a random amount of time, as computed by the function *delay*(), which returns a value uniformly between 0 and *MaximumTransmissionDelay*. The delay behavior is modelled in Stateflow using the *after* operator.

There is one *Channel_i* chart for each sequence number i in order to achieve concurrency when sending messages.

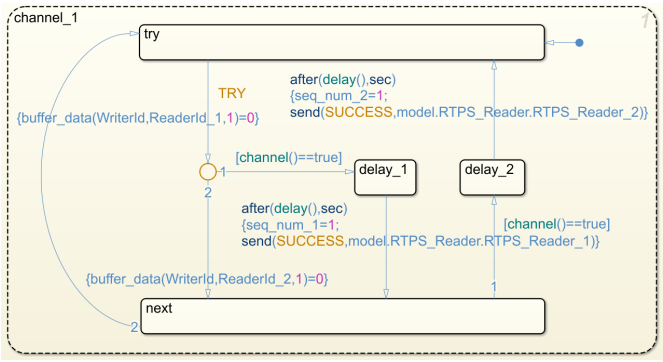

Fig.2. Stateflow model of channel for sending data.

Similarly, we can model the possibility of losing heartbeat or acknack messages using a channel model. Fig. 3 shows the case for heartbeat. It first receives the HEARTBEAT message from the writer, but only with probability computed by *channel*(), the message HEARTBEAT2 is successfully sent to the reader, after delaying a random amount of time as computed by *delay*().

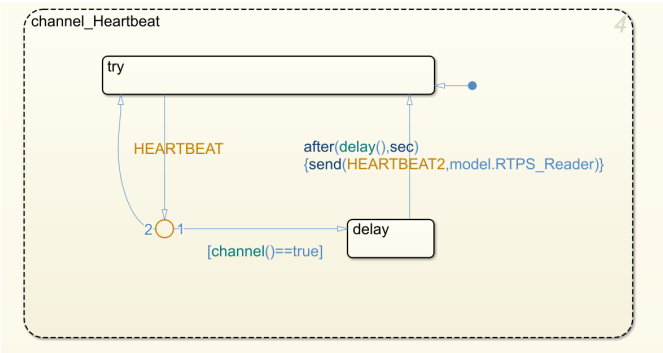

Fig.3. Stateflow model of channel for sending heartbeat.

4.3 RTPS Writer

As shown in Fig. 4, the behavior of the reliable StatefulWriter is described by an AND diagram consisting of two charts: *Writer_send* and *Writer_repair*.

The *Writer_send* chart handles sending data for the first time. It consists of three states: announcing, idle and pushing. Here pushing indicates the writer is in the process of sending data, announcing indicates the writer has finished sending the available data, and is periodically sending HEARTBEAT events, idle indicates that positive acknowledgements for all messages are received, hence the writer is idle. From the announcing state, the transition to pushing is triggered by the guard condition [*acked_all*() == false && *empty*() == false], indicating that there are some changes, represented by the array *unsent*, in the RTPS Writer HistoryCache that have not been sent to the RTPS Reader. The transition from announcing to idle is triggered by the guard condition [*acked_all*() == true], indicating that all changes are received and acknowledged by the reader and there are no new available changes in the RTPS Writer HistoryCache. Once [*acked_all*() == false] holds, the writer will return to the announcing state. On the pushing state, every time the condition [*cansend* == true] is satisfied, the unsent CacheChanges are sent by the TRY event in the order of sequence number and the values of *buffer_data* are updated by the function *set_message*(). The function *select_channel*() is used to choose the channel used to send messages by their sequence numbers. The transition from pushing to announcing is triggered by the guard condition [*empty*() == true] indicating that all changes in the RTPS Writer HistoryCache have been sent to the RTPS Reader. Note that this does not mean all changes have been received, only that there has been an attempt made to send all of them.

The *Writer_repair* chart handles re-sending data after negative acknowledgements. It consists of three states: waiting, must_repair and repairing. Here waiting is the default state, must_repair indicates that a negative acknowledgement is received, and data must be re-sent after some delay, repairing indicates the writer is in the process of re-sending data. The state en is an internal state of must_repair for correct modelling of timing constraints. On both the waiting and en state, there is a self-loop transition, triggered by

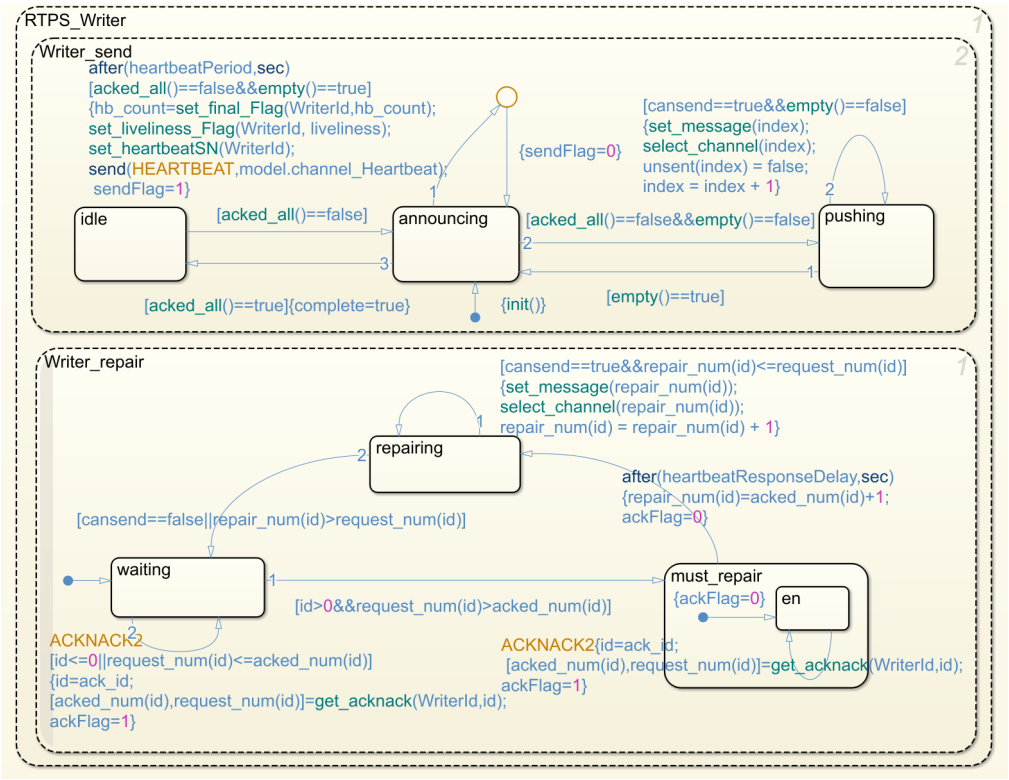

Fig.4. Stateflow model of RTPS writer.

the reception of ACKNACK2 from the RTPS Reader, and as a consequence, the Writer will update the local variables *acked_num* and *request_num* by the information in *buffer_acknack*. Once the guard condition [*request_num* > *acked_num*] is satisfied indicating that there are changes that have been requested by the RTPS Reader, it enters the must_repair state from the waiting state. The transition from must_repair to repairing is triggered by the firing of a timer indicating that a duration of *nackResponseDelay* has elapsed since the state must_repair was entered. On the repairing state, the writer will send the requested changes in the Writer's HistoryCache and update the values in *buffer_data* in the order of sequence number from the next of the *acked_num* to *request_num* as long as [*cansend* == true] is satisfied. If the repair action is completed, the guard condition [*repair_num* > *request_num*] is satisfied and it returns to the waiting state.

4.4 RTPS Reader

As shown in Fig. 5, the behavior of the Reliable StatefulReader is described by an AND diagram consisting of two charts: *Reader_receive* and *Reader_response*.

The *Reader_receive* chart has only one state ready for receiving messages from the RTPS Writer. Once the event SUCCESS arrives, the sequence number of the data/gap message is read by the Reader's HistoryCache from *buffer_data*, and *received_num* and *missing_num* are recomputed. If the event HEARTBEAT2 arrives, the variable *missing_num* is updated.

The *Reader_response* chart handles sending of acknack messages. It consists of three states: waiting, must_send_ack and may_send_ack. The waiting state is the initial state and the transition to other states is triggered by the reception of a HEARTBEAT2 event. The middle junction determines whether must_send_ack (indicating that acknack message should be sent in all

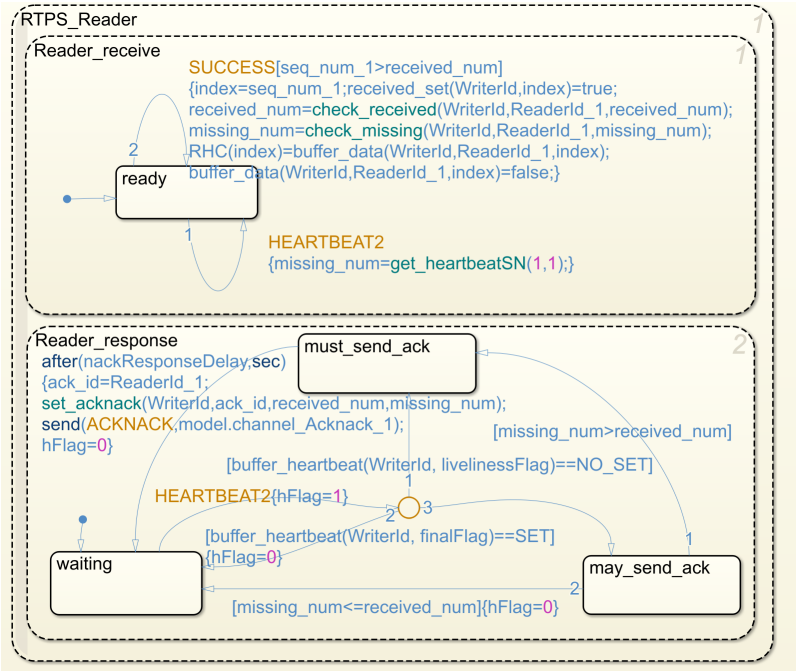

Fig.5. Stateflow model of RTPS reader.

cases) or may_send_ack (indicating that acknack message should be sent only if some messages are missing) should be entered. There are three different cases originating from the middle junction. If the FinalFlag in *buffer_heartbeat* is not set, then it enters must_send_ack. Otherwise, if the FinalFlag is set, and if the LivelinessFlag in the *buffer_heartbeat* is also set (indicating that the DDS DataWriter associated with the RTPS Writer of the message has manually asserted its liveliness using the appropriate DDS operation), it returns to the waiting state. The state may_send_ack is entered when the FinalFlag is set and the LivelinessFlag is not set. On the state must_send_ack, the Reader must send ACKNACK to the Writer after a delay defined by *nackResponseDelay*. When the transition executes, the acknack message is updated by setting *received_num* and *missing_num* in the *set_acknack* function, and the state waiting is entered. On the state may_send_ack, the transition to must_send_ack state is triggered by the guard condition [$missing\_num > received\_num$], and the transition to waiting state is triggered by the guard condition [$missing\_num \leq received\_num$] indicating that all available changes in the RTPS Writer are already received.

4.5 Implementation and Simulation

The entire Stateflow model for the RTPS protocol is composed of the above components. Note there is one copy of the *RTPS_Writer* diagram and *RTPS_Reader* diagram presented in Fig. 4 and Fig. 5 for each writer and reader, respectively, and one copy of channel diagram for each sequence number of data messages, as well as for the heartbeat and acknack messages.

For simulation, we consider the model including one RTPS Writer and two RTPS Readers. The list of data changes to be transmitted is stored in the array *cachechanges*, A *Publisher* sends data from *cachechanges* to the RTPS Writer at random time intervals by writing to the Writer's HistoryCache (WHC). Once the RTPS Writer receives the data, it sends the data to the matched RTPS Readers through the different channels. The RTPS Readers write received data

to their respective Reader's HistoryCache (RHC). If a data is initially available but becomes irrelevant later, a gap message represented by -1 in the *Data* field will be re-transmitted.

Table 2. Parameter setting

Parameter	Value
heartbeatPeriod	10 (s)
nackResponseDelay	10 (s)
heartbeatResponseDelay	5 (s)
PassingRate	0.5
MaximumTransmissionDelay	3 (s)

One simulation result is shown in Fig. 6. The simulation duration is set to 200 sec and the array *cachechanges* representing data to be transmitted is set to $[100, 200, 300, 400, 500]$. Other parameters of the model are set as in Table 2. From the simulation results, it can be seen that the two readers received data at different times, meaning that data is re-transmitted in case of loss. Finally, the RTPS Writer has received the *Acked_complete* signal indicating that the transmission of all available data has completed.

Finally, we use Simulink Coder to export C code from the Stateflow model. The result of execution of the exported program is in agreement with that of the above simulation.

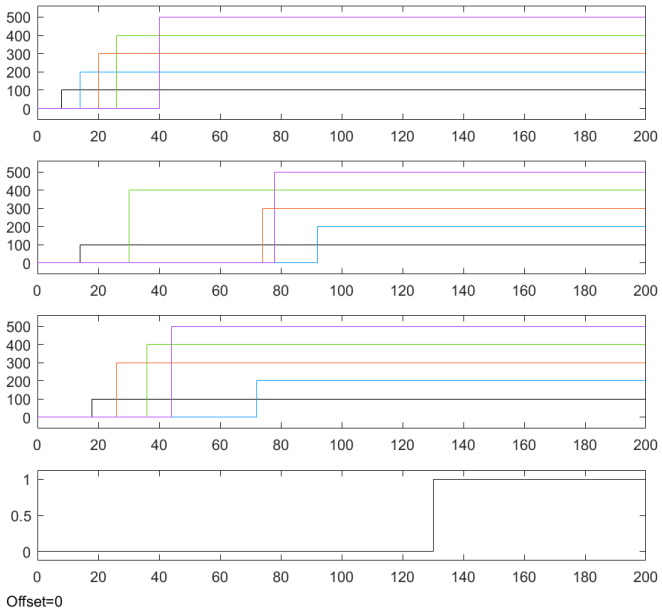

Fig.6. Results of simulation. (a) WHC; (b) RHC_1; (c) RHC_2; (d) Acked_complete signal. The lines in WHC indicate when each message is first sent to the Readers. The lines in RHC_1 and RHC_2 indicate when each message is first received by each Reader. The line in Acked_complete signal indicates the time when all messages are acknowledged by all readers.

5 Translation: From Stateflow Subset to TA in UPPAAL

In Section 4, we built the Stateflow model for the RTPS protocol and analyzed its behavior by simulation. In order to verify its correctness according to the RTPS specification, we will first translate the Stateflow model to an UPPAAL TA model, and then verify the translated TA model by model checking. We will prove the soundness of the translation that the original Stateflow model is a refinement of the translated TA model. As stated in Proposition 2 later, this implies that all properties in the universal fragment of untimed TCTL that hold for the TA model also hold for the Stateflow model.

Stateflow provides many complex features for modelling systems, some of which do not have correspondences in UPPAAL. In this work, we consider the translation of a limited subset of features, that is adequate for modelling real-time reactive systems such as the

RTPS protocol. Supported features include hierarchical states, junctions, transitions, actions including directed event broadcasting, and conditions including those expressed in temporal logic. More advanced features of Stateflow, such as supertransitions (transitions that cross multiple layers between states), early return logic and the event interrupt, are not considered. Especially, the execution of non-supertransitions will not cause the change of context. This makes both the translation and the translated TA much simpler, compared with [22], in which a Stateflow state with attached actions is translated to four cooperative parallel TAs.

Before presenting the translation rules, we introduce some auxiliary functions. Given a state S, we define en_S, dur_S, ex_S to represent the entry, during and exit actions attached to S respectively. Given a junction J, we define $isTer(J)$ to indicate whether J is a terminal junction. Given a state or junction S and transition with priority k, we define $cond(S,k)$ and $target(S,k)$ to be the condition and target of the transition respectively. During the execution, we maintain the following variables for a state or junction S: $source(S)$ is the source of the previous executed transition that leads to junction S, $sourceSta(S)$ is the nearest source state (not junction) in the previous transition path that leads to junction S, $entry(S)$ with default value 0 represents whether the entry action of state S is executed, and $clock(S)$ with default value 0 records the activation time of state S. We also introduce ACT to record the transition actions accumulated during the execution of a transition path, with initial value *skip*.

5.1 Translating Stateflow Charts

Both states and junctions exist in a Stateflow chart, so there are four cases for a transition: state to state, state to junction, junction to state, and junction to junction. We give the translation to TA for each case below. We first assume that the conditions of all outgoing transitions from a state or junction are disjoint, then this assumption is loosened at Section 5.1.4.

5.1.1 Translating a State

For a state S, assume there is an outgoing transition to a state S', labelled by (ev, c, ca, ta), where ev is the triggering event, c the condition, ca the condition action, and ta the transition action. In the translation, we map S to location U and S' to location U'. The TA transition from U to U' is defined with event ev (the event translation is explained in detail in the next part), guard $c \wedge entry(S)$, and an action list $ca; ex_S; (ACT; ta); clock(S') = 0; en_{S'}; entry(S') = 1; ACT = skip$. Here $entry(S')$ becomes true after the entry action of S' executes. The action list means that, when a state-to-state transition is enabled, the condition action will execute immediately, then the exit action of S executes, followed by the accumulated transition actions, and finally the target state is activated and the entry action of the target state executes. When entering a new state, the transition action ACT is reset to *skip*.

From U to U itself, we first add a transition to execute the entry action, with guard $\neg entry(S)$ and action $en_S; entry(S) = 1$, and then we add another transition to execute the during action, with guard $entry(S) \wedge \neg(cond(S,1) \vee \ldots \vee cond(S,n))$ and action $dur_S;\ clock(S) = clock(S) + 1$. It means that all of the outgoing transitions of S failed to execute, and the during action executes, the activation time of S increases by 1. The substates of S (if there is any) will be executed, which will be handled below.

If the transition is to a junction J, the translation is very similar to the first case, except that, the action list from U to U' will be changed to $ca; ACT = (ACT; ta); source(U') = U; sourceSta(U') = U$, i.e, the

transition path is still not complete, the source state is not exiting, and the transition actions will continue to be accumulated. The source of the transition and the source state leading to this junction are updated.

If there is no decomposition inside S, by applying the above translation process for each outgoing transition of S, a TA is obtained for state S. If there is decomposition C inside S, we first introduce a boolean variable $during(S)$ with default value 0 to indicate whether the during action is executed or not, and add guard $\neg during(S)$ and action $during(S) = 1$ to the during transition from U to U, and then we perform the following translation process:

1. Build the TA for the decomposition C, denoted by $\mathcal{U}_C$; meanwhile, denote the TA for the parent state S by $\mathcal{U}_S$.

2. Add a location I to $\mathcal{U}_S$, and add a transition from U to I, with event $in!$, condition $\neg(cond(S,1) \vee \cdots \vee cond(S,n)) \wedge during(S)$ indicating that the execution will go to the inside chart C after the during action executes; meanwhile, add a back transition from I to U. This means, the execution can always go back to the parent state, but it only makes sense when there is enabled outgoing transition for it;

3. For the outgoing transitions of U corresponding to the original ones of Stateflow, add event $out!$ to each of them. As a result, when some outgoing transition is enabled, the event $out!$ will broadcast to the inside chart C to transfer the execution back to S;

4. Add a location O to $\mathcal{U}_C$, and add a transition to each initial location of $\mathcal{U}_C$, with event $in?$ and action $during(S) = 0$. The reception of $in?$ will transfer the execution from outside to inside. For each location of $\mathcal{U}_C$, add a transition to O with event $out?$, whose reception will transfer the execution from inside to outside. Notice that the occurrence of $out!$ guarantees that some outgoing transition of S is enabled.

If S has decomposition, then the entry and exit actions need to be extended to include the decomposition, we omit the details here. In the following, for any state S, S_{de} denotes S and its decomposition, and $ex_{S_{de}}$ and $en_{S_{de}}$ the sequence of exit actions of S_{de} in the reverse order of entering the decomposition and the entry actions in the order of entering the decomposition.

By following the above translation process, we can obtain the TA for a state with decomposition inside.

5.1.2 Translating a Junction

For a junction J, assume there is an outgoing transition to a state S'. Similar to the previous case, assume the transition is labelled by (ev, c, ca, ta), and there are n outgoing transitions from J. In the translation, we map J to location U and S' to location U'. The TA transitions are defined similarly to the first case, except that: from U to U', the guard is simply c, and the action list is changed to be $ca; ex_{sourceSta(J)}; (ACT; ta); clock(S') = 0; en_{S'}; entry(S') = 1; ACT = skip$, meaning that the source state of current complete transition path will exit. Moreover, we add a back transition from U to its previous location *source(U)*, with guard $\neg(cond(J,1) \vee \cdots \vee cond(J,n)) \wedge \neg isTer(J)$. It means that if J is not terminal, and all of the outgoing transitions of J fail to execute, the execution backtracks to the previous state (or junction). If J is a terminal junction, then the execution will terminate at J. The two transitions from U to U are not needed anymore for the junction case.

If the transition is to a junction J', the translated TA is defined similarly to the third case,

except that from U to U', the action list is changed to be $ca; ACT = (ACT; ta); source(U') = U; sourceSta(U') = sourceSta(U)$.

5.1.3 Translating Conditions, Actions and Events

Above we define the translation of states and transitions for the four cases respectively. In most cases, the conditions and actions of Stateflow can be translated to UPPAAL TA conditions and actions directly. In particular, for the temporal logic conditions of Stateflow, there are two types: event-based and absolute-time based. We can express them in TA by introducing an extra counter to record the times that an event has taken, or a clock to record the time that has passed. For example, $after(T, sec)$ can be defined in TA by introducing a new clock variable as follows: for the source state, we introduce a new clock t and initialize it to be 0, then the transition with the condition $after(T, sec)$ is guarded by $t \geq T$, and meanwhile, we add the invariant $t \leq T$ for the source location.

For the directed event broadcast send(e, S), we represent it by a channel array $e[id_1..id_k]!$ in TA, where id_j for $1 \leq j \leq k$ are related indexes; in correspondence, for the receiving event e of Stateflow, it is represented by $e[id_1..id_k]?$ in TA.

Stateflow allows probabilistic behavior in actions, e.g, through function *channel*() in the chart *Channel* (see Section 4.2). These can be expressed by probabilistic transitions in TA. As an example, Fig. 7 gives the TA model of *Channel*, in which both the event broadcast and the probabilistic behavior are involved.

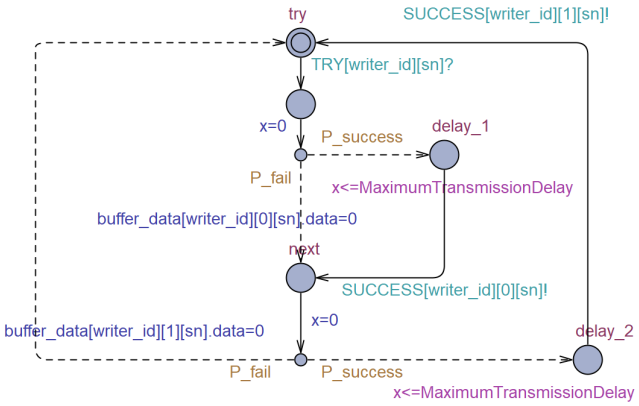

Fig.7. The timed automata translated from channel.

5.1.4 Translating States with Non-disjoint Transitions

The above translation preserves the priority order of the outgoing transitions of Stateflow states, but under the assumption that the transitions have disjoint conditions. We loosen this restriction here. Instead of representing events of Stateflow by event broadcasting in TA, we represent them by predicates. First of all, for each event e, a boolean global variable gv_e is introduced to represent whether it is available. Then the transitions are modified as follows. Take the first transition case as an example, assume it has priority number k, and there are n outgoing transitions from state S, then obviously $k \leq n$.

- If the transition from S to S' has no triggering event (ev is empty), then the guard from U to U' is changed to $c \wedge entry(S) \wedge \neg((cond(S,1) \wedge eventP(S,1)) \vee \cdots \vee (cond(S,k-1) \wedge eventP(S,k-1)))$, where $eventP(S,i)$ denotes the boolean variable corresponding to the event of transition i (if it exists, otherwise true by default). For each transition with priority i less than k, $cond(S,i) \wedge eventP(S,i)$ defines the overall premise for it to occur. The guard indicates that all transitions with higher priority are not enabled. The action list is same as before, if S is not sending an event.

- If S is sending an event e, then define the transition from U to U' with the same condition as above, but the action list of the previous one added by $gv_e = 1$ to make e available, and add another transition from U' to U' with guard gv_e and action $gv_e = 0$ to make event e unavailable.
- If S is receiving event, i.e. ev is not empty, change the guard from U to U' to be $c \wedge entry(S) \wedge gv_{ev} \wedge \neg((cond(S,1) \wedge eventP(S,1)) \vee \cdots \vee (cond(S,k-1) \wedge eventP(S,k-1)))$, and the action list is the same as before.
- From U to U itself, change the guard of the transition for executing the during action to be $\neg((cond(S,1) \wedge eventP(S,1)) \vee \cdots \vee (cond(S,n) \wedge eventP(S,n)))$.
- From a non-terminal junction to its source, the condition is strengthened in the same way.

Through this way of strengthening the guard for each outgoing transition, all transitions from locations of TA become disjoint, thus the priority order of transitions in the original Stateflow chart is preserved.

Example 1 In order to understand the translation process, we select the module *Writer_repair* in Fig. 4 for an illustration. It should be noticed that the outgoing transitions for the states are disjoint. Fig. 8 gives the translated TA for *Writer_repair*. First, we represent the four states waiting, must_repair, repairing and en by four corresponding locations. As state en is an internal state of state must_repair, it is translated to a separate TA. Locations inside and ini are added respectively to coordinate the execution between the two TAs.

The location must_repair broadcast events *in*! and *out*! in the transitions to locations inside and repairing respectively. On the other hand, in the TA of state en, there is a transition from location ini to en, with triggering event *in*?, and from en back to ini, there is a back transition with triggering event *out*?. For both waiting and en states, the self-loop transitions are triggered by event ACKNACK2. It is represented by the broadcast channel $ACKNACK2[wid][ack\_id]$? in the TA model, indicating the writer denoted by *wid* is waiting for receiving event from the reader denoted by *ack_id*. Similarly, at the repairing state, *select_channel*() plays the role of selecting the transmission channel and sending TRY event, which is represented by the broadcast channel $TRY[wid][repair\_num[id]]$! in the TA model.

5.2 Soundness of the Translation

Now we consider the soundness of the translation in the sense that the original Stateflow model is a refinement of the translated TA model. We first give the semantics of the Stateflow subset that is considered in the translation. Then we present the statement of the soundness theorem. The theorem is proved using a simulation-based argument. The semantics of Stateflow defined below is a formalization of a subset of the informal specification given in the document[2].

Semantics of a Stateflow subset First of all, we define the semantics of Stateflow states, which is a set of transition rules. Each configuration is (S, v), where S is a state or a junction, and v is a valuation which maps variables to their respective values. We use the same functions and symbols for states or junctions defined at the beginning of this section. The variables we record in v include the variables occurring in the Stateflow model, and the following auxiliary variables or arrays: gv_e, $during(S)$, $source(S)$, $sourceSta(S)$, ACT, and $clock(S)$ (same meaning as above). Given a state S, we define

[2]Stateflow User's Guide, 2016. http://www.mathworks.com/help/pdf_doc/stateflow.

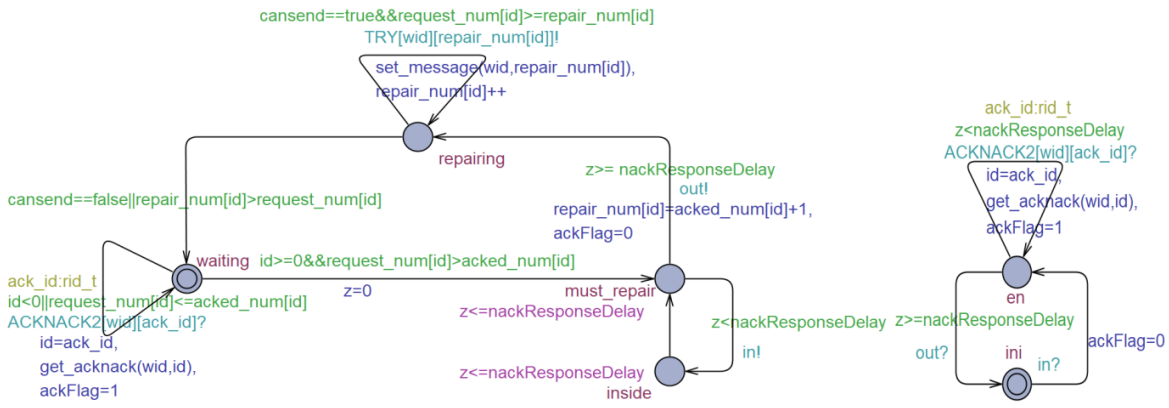

Fig.8. The TA Translated from Writer_repair

$trans(S,i)$ to represent the ith outgoing transition of S. Given a valuation v and an action a, we use $exec(a,v)$ to represent the resulting valuation after executing a from v. In particular, when a is a sending event, v sets gv_e to 1.

In the following, S denotes a state or junction, and it has $n \geq 0$ outgoing transitions.

(1) $(S,v) \xrightarrow{e} (t,v')$, if there exists $h > 0$ such that $trans(S,h) = (e,c,ca,ta)$ with target t, $v \in c$, $\neg(v \in (cond(S,1) \wedge eventP(S,1)) \vee \cdots \vee (cond(S,h-1) \wedge eventP(S,h-1)))$ (the transitions with higher priority than h are disabled), gv_e is 1, t is a junction, and $v' = exec(ca,v)[ACT \mapsto (v(ACT);tc), source(t) \mapsto S, sourceSta(t) \mapsto A]$, where A is S if S is a state, otherwise $sourceSta(S)$ if it is a junction. Notice that e can be empty.

This rule defines the case when the transition is enabled and executed, leading to a junction. As a result, the condition action executes, the transition action is put into the tail of ACT.

(2) $(S,v) \rightarrow (S,v')$, if there exists event e such that $v(gv_e) = 1$, and $v' = v[gv_e \mapsto 0]$. The event is disabled after it broadcasts to the chart.

(3) $(S,v) \xrightarrow{e} (t,v')$, if there exists $h > 0$ such that the conditions same to (1) case hold, except for that, t is a state, and $v' = exec(ca; ex_{A_{de}}; ACT; ta; en_{t_{de}}, v)[ACT \mapsto skip, clock(A_{de}) \mapsto 0]$, where A is S if S is a state, otherwise $sourceSta(S)$ if it is a junction. The clocks recording the activation times of the source state A and its decomposition are reset to 0.

This rule defines the case that a valid transition to a target state is performed, with the exit of the source state and all its active substates and the entry of the target state and its substates. The transition actions accumulated will be executed. For the new active state t, the transition action list is initialized to empty. The source state A_{de} and all the decomposition, and the junctions along the path become inactive, and in correspondence, t and its decomposition become active.

(4) $(S,v) \xrightarrow{1} (S,v')$, if $\neg(v \in (cond(S,1) \wedge eventP(S,1)) \vee \cdots \vee (cond(S,n) \wedge eventP(S,n)))$, $v(during(S))$ is 0, S is a state, and $v' = exec(dur,v)[during(S) \mapsto 1, clock(S) \mapsto clock(S)+1]$.

This rule indicates that, when there is no enabled

transition for a state S, then the during action of S will execute instead. Every time the during action is executed, the clock for recording the activation time of S will be increased by 1.

(5) $(S, v) \rightarrow (C, v[during(S) \mapsto 0])$, if $\neg(v \in (cond(S,1) \wedge eventP(S,1)) \vee \cdots \vee (cond(S,n) \wedge eventP(S,n)))$ and $v(during(S))$ is 1, and C is the outmost decomposition of S.

This rule indicates that, after the during action of S executes, the inside chart of S continues to execute. Notice that C may be an And diagram, whose transition rules are defined recursively below.

(6) $(S, v) \rightarrow (source(S), v)$, if $\neg(v \in (cond(S,1) \wedge eventP(S,1)) \vee \cdots \vee (cond(S,n) \wedge eventP(S,n)))$, and S is a non-terminal junction.

This rule indicates that, when there is no enabled transition for a non-terminal junction, then it will trace back to the previous state or junction. The un-tested transitions of the previous one will be executed. This can be guaranteed by introducing a boolean variable for each transition leading to a junction to indicate whether it is tested or not. We omit this detail for simplicity.

(7) $(S_1, S_2, v) \rightarrow (S_1', S_2', v')$, if there exist $(S_1, v) \xrightarrow{e} (S_1', v_1)$ and $(S_2, v) \rightarrow (S_2', v_2) \rightarrow (S_2', v_3)$ such that $v' = v_1 \uplus v_3$, $v_2(gv_e) = 1$ and $v_3 = v_2[gv_e \mapsto 0]$.

This rule defines the synchronization behavior based on directed event broadcasting. When a state (i.e, S_2) issues an event, it will broadcast to a state which is ready to receive this event. Normally the two states are parallel, and the variables are disjoint. The final state v' is the disjoint union of v_1 and v_3.

Based on the above transition rules for executing states, the semantics of a given Stateflow chart $\mathcal{D}$ can be defined. If it is an And diagram, written as $[S_1, \cdots, S_n]$, the configuration defines parallel states $(S_1, \cdots, S_n)$. The first rule defines the initialization, where all the parallel states and their decompositions are entered:

$$(S_1, \cdots, S_n, v_0) \rightarrow (S_1, \cdots, S_n, v_0')$$

where $v_0' = exec(en_{S_1 de}; \cdots; en_{S_n de}, v_0)$. The second rule defines the execution of states within one sample time:

$$(S_1, \cdots, S_n, v) \rightarrow (p_1', \cdots, p_n', v_n)$$

if there exist the transition paths for all S_i's:

$$(S_1, v) \rightarrow^* (p_1, v_1), \cdots, (S_n, v_{n-1}) \rightarrow^* (p_n, v_n)$$

where p_i has three cases: a state as a consequence of the first execution of the transition rule (3), or a terminal junction, or still S_i which has no other available transitions to take except for time progress. For the second case, p_i' is p_i (staying at the terminal junction), while for other cases, p_i' is the corresponding outmost active ancestor state. Notice that the entry actions of all S_i's are executed in a priority order. The same is true for the execution of these states.

The case for an Or diagram is similar to executing a state, we omit the detail here.

Until now, the semantics of a Stateflow chart $\mathcal{D}$ can be defined as a transition system $(Q, \rightarrow, Q^0, V)$, where $Q = (\mathcal{S}_1 \times \cdots \times \mathcal{S}_n) \times \mathcal{V}$ is the set of configurations for executing $\mathcal{D}$, $\mathcal{S}_i$ the set of states of S_i, $Q_0 = \{(S_1, \cdots, S_n, v_0)\}$ is the initial configuration. V is the set of variables of $\mathcal{D}$, and $\mathcal{V}$ the set of valuations that map variables in V to their values. For each given configuration q, let $q.val$ denote the valuation function of the configuration q.

Soundness Below we state the soundness of the translation as a refinement relation between the two transition systems. Let Q_1, Q_2 be two sets of configurations, and $\rightarrow_1$, $\rightarrow_2$, V_1, V_2 the transition relations and the variables for Q_1 and Q_2 respectively, and $V = V_2 \cap V_1$. Let $q.val|_V$ denote the projection of the valuation of q to the variable set V.

Definition 1 (Simulation) *Let $T_i = (Q_i, \rightarrow_i, Q_i^0, V_i)$ for $i = 1, 2$ be two transition systems. A relation $\mathcal{B} \subseteq Q_1 \times Q_2$ is a simulation relation from T_1 to T_2, iff for all $(q_1, q_2) \in \mathcal{B}$, we have (1) $q_1.val|_V = q_2.val|_V$, and (2) for any $q_1 \rightarrow_1 q_1'$, there exists a transition $q_2 \rightarrow_2 q_2'$ such that $(q_1', q_2') \in \mathcal{B}$.*

We say that T_1 is a refinement of T_2, written $T_1 \leq T_2$, if there exists a simulation relation from T_1 to T_2 according to the above definition.

Theorem 1 (Soundness of the translation)**.** *Let $\mathcal{D}$ be a Stateflow chart, and $\mathcal{U}$ be the TA obtained from $\mathcal{D}$ by applying our translation. Assume $T_i = (Q_i, \rightarrow_i, Q_i^0, V_i)$ for $i = 1, 2$ are the transition systems of $\mathcal{D}$ and $\mathcal{U}$, with the same initial valuation for variables in $V_1 \cap V_2$. Then there exists a simulation relation $\mathcal{B} \subseteq Q_1 \times Q_2$ such that, for any $q_1^0 \in Q_1^0$, there exists $q_2^0 \in Q_2^0$ satisfying $(q_1^0, q_2^0) \in \mathcal{B}$.*

The proof is by explicitly constructing a simulation relation between $\mathcal{D}$ and $\mathcal{U}$, which means for each transition in $\mathcal{D}$, find a corresponding transition in $\mathcal{U}$ resulting in equivalent evaluations over common variables. The details are given in the appendix.

5.3 Preservation of Properties

According to the semantics of Stateflow, we obtain the following fact.

Proposition 1. *Let $\mathcal{D}$ be a Stateflow chart. Assume its transition system is $T = (Q, \rightarrow, Q^0, V)$, then if $\mathcal{D}$ contains no terminal junction, the transition relation $\rightarrow$ is total, i.e. for every configuration $C \in Q$, there is $C' \in Q$ such that $(C, C') \in \rightarrow$ is enabled.*

The requirement that the transition relation $\rightarrow$ is total is necessary for the preservation of properties. It ensures that a path of a Stateflow chart corresponds to a full path of the translated TA, and not a proper prefix of the path. The proof is given in the appendix.

Based on the above fact and Theorem 1, we can further get the following result.

Proposition 2. *Let φ be a property in the universal fragment of untimed TCTL and $\mathcal{D}$ is a Stateflow chart without terminal junction. If all variables occurring in φ are in V, then T_2 satisfies φ implies T_1 satisfies φ. Here $\mathcal{D}, T_1, T_2, V$ are as defined in Theorem 1.*

This is a direct result of the preservation theorem of the classical abstraction refinement framework [26].

Discussion Above we prove the preservation of properties in the general refinement framework, i.e. provided two transition systems satisfy the refinement relation, the concrete model has a total transition relation, and the property is in the universal fragment, then the property proved for the abstract model is preserved for the concrete model. But in fact, the TA translated from the Stateflow model according to our translation process satisfy stronger property than refinement: each transition of the TA corresponds to some one step execution of the original Stateflow model. They are not equivalent only because the TA does not keep the priority order of the execution of parallel states of the Stateflow model. However, in most cases, parallel states do not share variables in Stateflow, e.g. the model for the RTPS protocol in this paper. Therefore, more properties other than the universal fragment of the TCTL are preserved for the Stateflow model. We will leave the formal justification of these for future work.

6 Verification in Uppaal

Using the above translation method, we transform the whole Stateflow model of the RTPS protocol into Uppaal timed automata. Fig. 8 shows the result for one module as an illustration. In this section, we describe the verification and performance estimation of the model in Uppaal.

First, we verify that the TA model is deadlock-free using the TCTL formula `A[] not deadlock`. Since this formula is not in the universal fragment of TCTL, it does not automatically carry over to the Stateflow model. All TCTL properties shown below are in the universal fragment of untimed TCTL, and hence by Proposition 2 also hold for the Stateflow model.

Consistency and sequentiality A writer must send out data samples in the order they were added to its HistoryCache and the readers will get the messages in the same order of the sequence number in their HistoryCache.

- Changes in the writer's HistoryCache are eventually added from the matched DDS Writer in order.

```
A<> forall(sq:sq_t)
WHC[sq]==cachechanges[sq]
```

- Once the RTPS Writer receives acknowledgement for all messages, the data is consistent between the writer and the matched readers, and the order of the data is the same as in the DDS Writer.

```
A[] forall(id:rid_t) (forall(sq:
    sq_t)
acked_num[id]==sq
imply WHC[sq]==cachechanges[sq]
and RHC[id][sq]==cachechanges[sq
    ])
```

Heartbeat and acknowledgement mechanism A writer must periodically inform each matching reader of the availability of data samples by sending a HEARTBEAT message.

- When the transmission is not complete, the RTPS Writer always sends HEARTBEAT messages with a period of *Heartbeat_Period*. The function *acked_all*() is used to indicate the acknowledged information from all matched Readers and identify whether the transfer is complete. The variable *sendFlag* is used to record the sending of HEARTBEAT and x is the periodic clock. $sendFlag == 1$ means the HEARTBEAT message is sending at that time, otherwise $sendFlag == 0$.

```
Writer_send(WriterId).x>
    Heartbeat_Period
and !acked_all()
--> Writer_send(WriterId).
    sendFlag==1
```

- When receiving an ACKNACK message indicating a reader is missing some data samples, the writer must enter the must_repair state to eventually respond by sending the missing data samples. Here the variable *ackFlag* identifies the signal to be repaired sent by the readers.

```
A<> Writer_repair(WriterId).
    ackFlag==1
and !acked_all()
imply Writer_repair(WriterId).
    must_repair
```

- Upon receiving a HEARTBEAT message with final flag not set, the reader must enter must_send_ack state to eventually respond with an ACKNACK message. Here *hFlag* indicates that the reader received the HEARTBEAT message.

```
A<> Reader_response(ReaderId).
    hFlag==1
and buffer_heartbeat[WriterId].
    finalFlag==NO_SET
imply Reader_response(ReaderId).
    must_send_ack
```

- After receiving a HEARTBEAT while a sample is missing, the reader must enter must_send_ack

```
A[] not deadlock
A<> forall(sq:sq_t) WHC[sq]==cachechanges[sq]
A[] forall(id:rid_t) (forall(sq:sq_t) acked_num[id]==sq imply WHC[sq]==cachechanges[sq] and RHC[id][sq]==cachechanges[sq])
Writer_send(WriterId).x>Heartbeat_Period and !acked_all() --> Writer_send(WriterId).sendFlag==1
A<> Writer_repair(WriterId).ackFlag==1 and !acked_all() imply Writer_repair(WriterId).must_repair
A<> Reader_response(ReaderId).hFlag==1 and buffer_heartbeat[WriterId].finalFlag==NO_SET imply Reader_response(ReaderId).must_send_ack
A<> Reader_response(ReaderId).hFlag==1 and has_missing_sample(ReaderId) imply Reader_response(ReaderId).must_send_ack
A[] forall(sq:sq_t) is_acked_by_all(sq) imply forall(id:rid_t) RHC[id][sq]==WHC[sq]
A[] acked_all() imply forall(id:rid_t) RHC[id]==cachechanges
```

Fig.9. Verification results.

state to eventually respond with an ACKNACK message. The function *has_missing_sample*() is used to judge whether some data is missing at present.

```
A<> Reader_response(ReaderId).
    hFlag==1
and has_missing_sample(ReaderId)
imply Reader_response(ReaderId).
    must_send_ack
```

- Therefore, the acknowledgement mechanism guarantees that once the acknowledged number equals the last sequence number sent to matched readers, all changes in the readers' HistoryCache are the same as in writer's HistoryCache, that is,

```
A[] forall(sq:sq_t)
    is_acked_by_all(sq)
imply
forall(id:rid_t) RHC[id][sq]==WHC
    [sq]

A[] acked_all() imply
forall(id: rid_t) RHC[id]==
    cachechanges
```

where the functions *is_acked_by_all*() and *acked_all*() are used to indicate the acknowledged information from all matched Readers.

The verification results for all of the above properties are shown in Fig. 9, in which green circles indicate that the corresponding property is proved to hold for the TA model.

Statistical Model Checking (SMC) is concerned with running a sufficient number of simulations of a stochastic system model to obtain statistical evidence with a predefined level of confidence of the property to be checked. This offers advantages over probabilistic model checking as it does not need to traverse the whole state space of the system, and so scales up to more complex systems and properties. In our case, we perform the query `Pr[<=300;1000] (<> acked_all())`, which estimates the probability that the given input data can be successfully transmitted and the acknowledgement signals are received by the function *acked_all*() in a fixed amount of time (300s). Each query is performed with 1000 runs. Starting from the choice of parameters of the model in Section 4.5, we perform experiments with variations of different parameters one at a time.

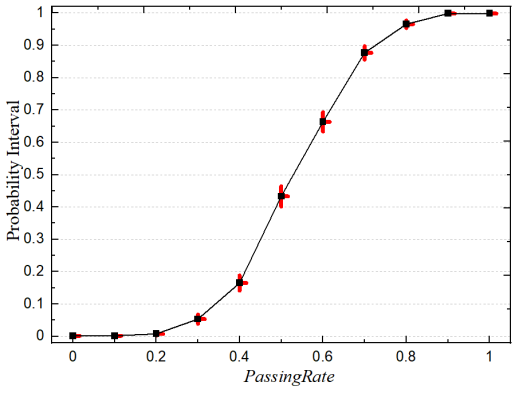

Fig.10. The probability for different passing rates with 95% confidence interval.

First, we perform the simulation with different passing rates in *Channel* ranging from 0 to 1. The relationship between passing rate and the probability that the query holds is shown in Fig. 10. UPPAAL SMC reports the probability interval with 95% confidence. It is clear

that the probability of successful transmission within the time bound increases as the passing rate increases. When the passing rate exceeds 0.8, data transmission is almost guaranteed to complete.

Next, we test the effect of different timing parameters on the transmission efficiency of the model. In particular, we consider the influence of timing parameters *nackResponseDelay* and *heartbeatResponseDelay* on the model when the channel quality ($PassingRate = 0.7$) and heartbeat period ($heartbeatPeriod = 10$) are fixed. Other model settings also remain the same. As shown in Fig. 11, different timing parameter settings lead to different probability that the query holds. Fig. 11(a) shows the probabilities when *heartbeatResponseDelay* changes from 0 to 40 with *nackResponseDelay* set to 5, 10 and 20 respectively. Fig. 11(b) shows the probabilities when *nackResponseDelay* changes from 0 to 40 with *heartbeatResponseDelay* set to 5, 10 and 20 respectively. It can be seen that smaller delay leads to more efficient data transmission in the formal model. On the other hand, a small delay may cause network congestion in actual deployment of the system, so we should choose the appropriate timing parameter to improve the model without seriously affecting the transmission efficiency.

Note that we do not consider probability in the refinements, hence these quantitative properties cannot be automatically transferred to the Stateflow model. However, from the discussion of Section 5.3, plus the analysis of the concrete Stateflow model, we infer that both the deadlock freedom property and the quantitative properties also hold for the Stateflow model. But the formal guarantee is not given here.

7 Conclusions

This paper presents formal models of the RTPS (Real-Time Publish and Subscribe) protocol using Simulink/Stateflow and UPPAAL. The Simulink/Stateflow model is used for simulation and code generation. We check that the results of the simulation are as expected from the specification, and agree with the behavior of the generated code. The UPPAAL model is used to verify the important properties of the protocol by model checking. Statistical model checking in UPPAAL is used to estimate the performance of the model as a function of different choices of parameters. The translation rules from Simulink/Stateflow to timed automata are defined and the soundness of the translation (stated as a refinement relation) is proved. Our work builds a complete framework of modelling, simulation and verification of the RTPS protocol.

Currently, we only handle limited aspects of the RTPS protocol. Possible future extensions include taking into consideration of message fragments and the discovery module. This will make the model more complex, likely beyond the model-checking capabilities of UPPAAL. Hence, verifying properties of the extended model will be a major challenge. Finally, translation from Simulink/Stateflow to timed automata is currently manual. The next step is to implement an automatic translation tool, and add other advanced features of Simulink/Stateflow.

References

[1] Andres F, Boulos J. DDS: the data delivery service. In *IFIP World Conference on IT Tools.*, January 1996, pp.487-494.

[2] Hugues J, Pautet L, Kordon F. A Framework for DRE middleware, an Application to DDS. In *Proc. Ninth IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing*, May 2006, pp.224-231.

[3] Al-Madani B. RTPS Middleware for Real-Time Distributed Industrial Vision Systems. In *Proc.*

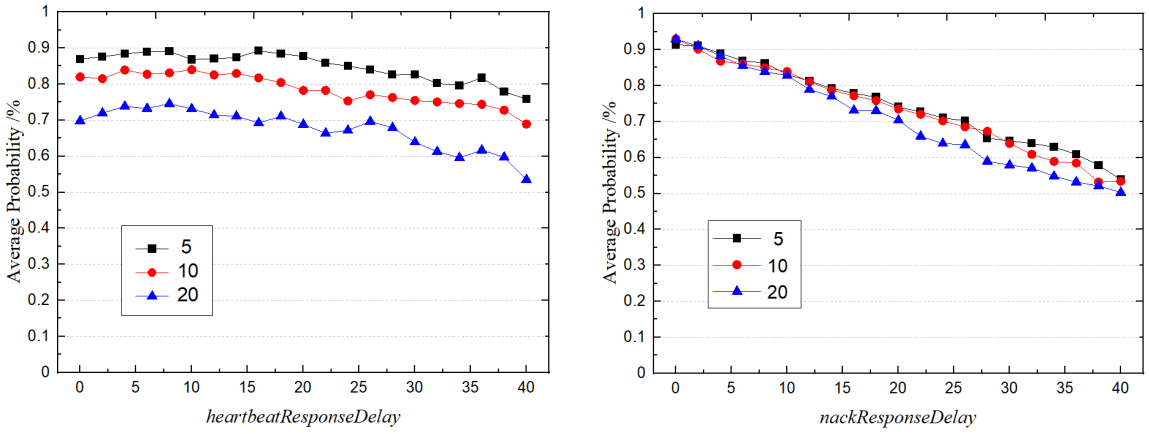

Fig.11. The average probability with 95 % confidence. (a) heartbeatResponseDelay with nackResponseDelay set to 5, 10 and 20 respectively; (b) nackResponseDelay with heartbeatResponseDelay set to 5, 10 and 20 respectively.

Embedded and Real-Time Computing Systems and Applications Conference, September 2005, pp.361-364.

[4] Al-Madani B, Al-Saeedi M, Al-Roubaiey A. Scalable Wireless Video Streaming over Real-Time Publish Subscribe Protocol (RTPS). In *Proc. 17th IEEE/ACM International Symposium on Distributed Simulation and Real Time Applications*, October 2013, pp. 221-230.

[5] Behrmann G, David A, Larsen K G. A Tutorial on UPPAAL. In *Proc. of Formal Methods for the Design of Real-Time Systems, LNCS 3185*, 2004, pp. 200-236.

[6] David A, Larsen K G, Legay A, Mikucionis M, Poulsen D B. UPPAAL SMC tutorial. *International Journal on Software Tools for Technology Transfer*, 2015, 17(4): 397-415.

[7] Beckman K, Reininger J. Adaptation of the DDS security standard for resource-constrained sensor networks. In *International Symposium on Industrial Embedded Systems*, June 2018, pp.1-4.

[8] Youssef T A, Hariri M E, Elsayed A T, Mohammed O A. A dds based energy management framework for small microgrid operation and control. *IEEE Trans. Industrial Informatics*, 2018, 14(3):958-968.

[9] Perez H, Gutierrez J J. Modelling the QoS parameters of DDS for event driven real-time applications. *Journal of Systems and Software*, 2015, 104(6):126-140.

[10] Alaerjan A, Kim D, Kafaf D A. Modeling functional behaviors of DDS. In *Proc. of IEEE SmartWorld*, 2017, pp.1-7.

[11] Anneke K. Object Constraint Language: Metamodeling Semantics. In *UML 2 Semantics and Applications.*, 2009, pp.163-178.

[12] Liu Y, Guan Y, Li X, Wang R, Zhang J. Formal analysis and verification of DDS in ROS2. In *Proc. of International Conference on Formal Methods and Models for System Design*, October 2018, pp.62-66.

[13] Yin J, Zhu H, Fei W, Xu Q, Wu R. Formalization and Verification of RTPS StatefulWriter Module

Using CSP. In *Proc. of International Conference on Software Engineering and Knowledge Engineering*, July 2019, pp.147-198.

[14] Brookes S D, Hoare C A R, Roscoe A W. A theory of communicating sequential processes. *Journal of the ACM*, 1984, 31(3):560-599.

[15] Hoare C A R. Communicating Sequential Processes. Prentice-Hall, 1985.

[16] Yang Y, Jiang Y, Gu M, Sun J. Verifying simulink stateflow model: timed automata approach. In *Proc. of IEEE/ACM International Conference on Automated Software Engineering*, 2016, pp.852-857.

[17] Kang E, Ke L, Hua M, Wang Y. Verifying Automotive Systems in EAST-ADL/Stateflow Using UPPAAL. In *Proc. of Software Engineering Conference, IEEE*, 2016, pp.143-150.

[18] Hamon G, Rushby J. An operational semantics for Stateflow. *International Journal on Software Tools for Technology Transfer*, 2007, 9(5-6):447-456.

[19] Tiwari A. Formal semantics and analysis methods for Simulink/Stateflow models. Technical Report, SRI International. `http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.1.3492`, Mar. 2002.

[20] Zou L, Zhan N, Wang S, Franzle M. Formal verification of Simulink/Stateflow diagrams. In *Proc. of Automated Technology for Verification and Analysis, LNCS vol. 9346, Springer*, 2015, pp.464-481.

[21] Chen C, Sun J, Liu Y. Formal modeling and validation of Stateflow diagrams. *International Journal on Software Tools for Technology Transfer*, 2012, 14(6):653-671.

[22] Rajeev A, David L D. A Theory of Timed Automata. *Theoretical Computer Science*, 1994, 126(2):183-235.

[23] Berhmann G, David A, Larsen K G, Pettersson P, Yi W. Developing UPPAAL over 15 years. *Software - practice and Experience*, 2011, 41(2):133-142.

[24] Younes H L S, Kwiatkowska M, Normaln G, Parker D. Numerical vs. statistical probabilistic model checking. *International Journal on Software Tools for Technology Transfer*, 2007, 15(11):1427-1434.

[25] Georgios E F, George J P. Robust Sampling for MITL Specifications. In *Proc. of Formal Modeling and Analysis of Timed Systems*, Oct. 2007, pp.147-162.

[26] Grumberg O, Long D E. Model checking and modular verification. *ACM Transactions on Programming Languages and Systems*, 1994, 16(3):843-871.

First Author [Qianqian Lin] Qianqian Lin received her Bachelor's degree in Electrical and Information Engineering from University of Science and Technology in China in 2017. She is currently a master student at Institute of Software, Chinese Academy of Sciences. Her research interests are modelling and simulation of embedded systems.

Second Author [Shuling Wang] Shuling Wang received her Bachelor's degree in Mathematics from Lanzhou University in 2003, and Ph.D. degree in Computer Science from Peking University in 2008. She worked as a postdoc at United Nations University - International Institute for Software Technology from 2008 to 2010, and at Institute of Software, Chinese Academy of Sciences from 2010 to 2012. Since 2017, she is an associate research professor at Institute of Software, Chinese Academy of Sciences. Her research interests are formal methods related to embedded systems, including formal modelling, verification, and code generation.

Third Author [Bohua Zhan] Bohua Zhan received his Bachelor's degree in Mathematics from Massachusetts Institute of Technology in 2010, and Doctorate in Mathematics from Princeton University in 2014. He worked as a postdoc at Massachusetts Institute of Technology from 2014 to 2017, and at Technical University of Munich from 2017 to 2018. Since 2018, he is an associate research professor at Institute of Software, Chinese Academy of Sciences. His research interests are formal methods, in particular interactive theorem proving and cyber-physical systems.

Fourth Author [Bin Gu] Bin Gu received his Master's degree in 1994 and Ph.D. degree in 2020 from Harbin Institute of Technology and Northwest Polytechnical University respectively. He is currently a professor at Beijing Institute of Control Engineering. His research interests are embedded systems and dependable softwares.

Appendix

In the appendix, we first present the more complete semantics of TA, then the proof of the soundness of the translation from Stateflow to TA.

8 Semantics of TA

Formally, a timed automaton is a tuple (L, l_0, C, A, E, I), where L is a set of locations, $l_0 \in L$ the initial location, C the set of clocks, A the set of actions, co-actions and the internal τ-action, $E \subseteq L \times A \times B(C) \times 2^C \times L$ the set of edges between locations with an action, a guard and a set of clocks to be reset, and $I : L \to B(C)$ assigns invariants to locations. The semantics can be defined as a labelled transition system $\langle S, s_0, \to \rangle$, where $S \subseteq L \times \mathbb{R}^C$ is the set of states, $s_0 = (l_0, u_0)$ is the initial state, and $\to \subseteq S \times (\mathbb{R}_{\geq 0} \cup A) \times S$ is the transition relation such that

- $(l, u) \xrightarrow{d} (l, u + d)$ if $\forall d' : 0 \leq d' \leq d \Rightarrow u + d' \in I(l)$,
- $(l, u) \xrightarrow{a} (l', u')$ if there exists $e = (l, a, g, r, l') \in E$ $s.t.$ $u \in g, u' = [r \mapsto 0]u$ and $u' \in I(l')$,

where for $d \in \mathbb{R}_{\geq 0}$, $u + d$ maps each clock x in C to the value $u(x) + d$, and $[r \mapsto 0]u$ denotes the clock valuation which maps each clock in r to 0 and agrees with u over $C \setminus r$.

The TAs can be composed into a network of timed automata over a common set of clocks and actions, consisting of n TAs $A_i = (L_i, l_0^i, C, A, E_i, I_i), 1 \leq i \leq n$. Let $\bar{l}_0 = (l_1^0, \ldots, l_n^0)$ be the initial location vector. The semantics is defined as a transition system $\langle S', s_0, \rightarrow \rangle$, where $S' = (L_1 \times \cdots \times L_n) \times \mathbb{R}^C$ is the set of states, $s_0' = (\bar{l}_0, u_0)$ is the initial state, and $\rightarrow \subseteq S' \times S'$ is the transition relation defined by:

- $(\bar{l}, u) \xrightarrow{d} (\bar{l}, u + d)$, if $\forall d'. 0 \leq d' \leq \Rightarrow u + d' \in I(\bar{l})$,
- $(\bar{l}, u) \xrightarrow{a} (\bar{l}[l_i'/l_i], u')$, if there exists $l_i \xrightarrow{\tau g r} l_i'$ *s.t.* $u \in g, u' = [r \mapsto 0]u$ and $u' \in I(\bar{l}[l_i'/l_i])$,
- $(\bar{l}, u) \xrightarrow{a} (\bar{l}[l_i'/l_i, l_j'/l_j], u')$ if there exist $l_i \xrightarrow{c?g_i r_i} l_i'$ and $l_i \xrightarrow{c!g_j r_j} l_i'$, s.t. $u \in (g_i \wedge g_j), u' = [r_i \cup r_j \mapsto 0]u$ and $u' \in I(\bar{l}[l_i'/l_i, l_j'/l_j])$.

9 Soundness proof

Theorem 1 (Soundness of translation) *Let $\mathcal{D}$ be a Stateflow chart and $\mathcal{U}$ be the TA obtained from $\mathcal{D}$ according to our translation. Assume $(Q_i, \rightarrow_i, Q_i^0, V_i)$ for $i = 1, 2$ are the transition systems of $\mathcal{D}$ and $\mathcal{U}$, with the same initial valuation for variables in $V_1 \cap V_2$. Then there exists a simulation relation $\mathcal{B} \subseteq Q_1 \times Q_2$ such that, for any $q_1^0 \in Q_1^0$, there exists $q_2^0 \in Q_2^0$ satisfying $(q_1^0, q_2^0) \in \mathcal{B}$.*

Proof According to the translation given in Section 5.1, for each state vector $(S_1, \cdots, S_n)$ in $\mathcal{D}$, there is a location vector $(U_1, \cdots, U_n)$ defined in $\mathcal{U}$ (each denoted by U_{S_i} in the following). We use q_i to represent state vectors in the following. We construct the simulation relation $\mathcal{B}$ between $\mathcal{D}$ and $\mathcal{U}$ as follows. First of all, add $\{q_1^0, U_{q_1^0}\}$ where $q_1^0 \in Q_1^0$, in fact $q_1^0 = (S_1, \cdots, S_2, v_0)$ for some initial v_0. Obviously, $U_{q_1^0} \in Q_2^0$.

Consider the initialization transition of $\mathcal{D}$, i.e. $([S_1, \cdots, S_n], v_0) \rightarrow ([S_1, \cdots, S_n], v_0')$ where $v_0' = exec(en_{S_{1de}}; \cdots; en_{S_{nde}}, v_0)$. For ease of presentation, below we will use en_{S_1} and ex_{S_1} to represent $en_{S_{1de}}$ and $ex_{S_{1de}}$ resp., i.e. the entry and exit actions are always referring to the ones of a state and its decomposition. According to our translation, in the translated TA $\mathcal{U}$, for each U_{S_i}, there is a transition

$$U_{S_i} \xrightarrow{\neg entry(S_i), en_{S_i}; entry(S_i)=1} U_{S_i}$$

enabled, as in the beginning, $entry(S_i)$ is 0. By taking the one-step transitions of all corresponding U_{S_i} for all i, we get the resulting location vector and the valuation. The resulting state and location vector are still the same as initial ones, and the valuations also satisfy the requirement, since parallel states do not write the same variables. Thus the resulting configurations are still in $\mathcal{B}$.

Now consider the more complex transition that corresponds to the execution of states within one sample time step. For some state S among the parallel states, if $(S, v) \rightarrow^* (p, v')$, there are four cases (the following assume a general setting that the Stateflow states have non-disjoint transitions and have decomposition inside. The simpler case can be proved easily).

- If p is a state by taking transition (3), there exists $h > 0$ such that $trans(S, h) = (e, c, ca, ta)$ with target p, $v \in c$, $\neg(v \in (cond(S, 1) \wedge eventP(S, 1)) \vee \cdots \vee (cond(S, h-1) \wedge eventP(S, h-1)))$ (the transitions with higher

priority than h are disabled), gv_e is 1, and $v' = exec(ca; ex_A; ACT; ta; en_p, v)[ACT \mapsto skip, clock(A) \mapsto 0]$, where A is S if S is a state, otherwise $sourceSta(S)$ if it is a junction. In the translated TA, there is a corresponding transition

$$U_S \xrightarrow{c\wedge entry(S)\wedge gv_e\wedge\neg((cond(S,1)\wedge eventP(S,1))\vee\cdots\vee(cond(S,k-1)\wedge eventP(S,k-1))),ca;ex_A;(ACT;ta);en_p;entry(p)=1;ACT=skip} U_p.$$

Obviously, the resulting valuations are equal over the common variables as they execute the same actions. Thus the targets are in $\mathcal{B}$.

- If p is a state without available outgoing transitions, then before reaching p, S tried all its outgoing transitions, execute the during transitions and its decomposition (if there is any). We prove that for each transition of S, there is a corresponding transition of U_S.

  If the transition (1) executes, there exists $h > 0$ such that $trans(S, h) = (e, c, ca, ta)$ with target t, $v \in c$, $\neg(v \in (cond(S, 1) \wedge eventP(S, 1)) \vee \cdots \vee (cond(S, h-1) \wedge eventP(S, h-1)))$ (the transitions with higher priority $< h$ are disabled), gv_e is 1, t is a junction, and $v' = exec(ca, v)[ACT \mapsto (v(ACT); tc), source(t) \mapsto S, sourceSta(t) \mapsto A]$, where A is S if S is a state, otherwise $sourceSta(S)$ if it is a junction. In the translated TA, there is a corresponding transition:

$$U_S \xrightarrow{c\wedge entry(S)\wedge gv_e\wedge\neg((cond(S,1)\wedge eventP(S,1))\vee\cdots\vee(cond(S,h-1)\wedge eventP(S,h-1))),\ ca;ACT=(ACT;ta);source(U_t)=U_S;sourceSta(U_t)=sourceSta(U_S)} U_t.$$

  Obviously, the resulting valuations are equal over the common variables.

  If transition (2) executes, in the translated TA, there is a corresponding transition

$$U_S \xrightarrow{gv_e,gv_e=0} U_S.$$

  The transition (3) will not happen for the case when the final state p is a state without available outgoing transitions.

  If transition (4) executes, in the translated TA, there is a corresponding transition:

$$U_S \xrightarrow{\neg during(S)\wedge\neg((cond(S,1)\wedge eventP(S,1))\vee\cdots\vee(cond(S,n)\wedge eventP(S,n))),dur_S;during(S)=1;clock(S)=clock(S)+1} U_S.$$

  If transition (5) executes, i.e. $(S, v) \to (C, v[during(S) \mapsto 0])$, if $\neg(v \in (cond(S, 1) \wedge eventP(S, 1)) \vee \cdots \vee (cond(S, n) \wedge eventP(S, n)))$ and $v(during(S))$ is 1, and C is the outmost decomposition of S. In the translated TA, there is a corresponding transition:

$$(U_S, O) \xrightarrow{\neg((cond(S,1)\wedge eventP(S,1))\vee\cdots\vee(cond(S,n)\wedge eventP(S,n)))\wedge during(S),during(S)=0} (I, U_C)$$

  where

$$U_S \xrightarrow{in!,\neg((cond(S,1)\wedge eventP(S,1))\vee\cdots\vee(cond(S,n)\wedge eventP(S,n)))\wedge during(S)} I, O \xrightarrow{in?,during(S)=0} U_C$$

hold. By hiding I and U, the transition is consistent with the Stateflow transition.

If transition (6) executes, in the translated TA, there is a corresponding transition:

$$U_S \xrightarrow{\neg isTer(J)\wedge\neg((cond(S,1)\wedge eventP(S,1))\vee\cdots\vee(cond(S,n)\wedge eventP(S,n)))} source(U_S).$$

If transition (7) executes, then first by induction, there are corresponding transitions $(U_{S_1}, w) \xrightarrow{gv_e} (U_{S_1'}, w_1)$ and $(U_{S_2}, w) \xrightarrow{gv_e=1} (U_{S_2'}, w_2) \xrightarrow{gv_e, gv_e=0} (U_{S_2'}, w_3)$. Then according to the semantics of TA, we have $(U_{S_1}, U_{S_2}, w) \rightarrow (U_{S_1'}, U_{S_2'}, w')$ such that $w' = w_1 \uplus w_3$. The disjoint case with event broadcasting can be proved easily.

All the above cases produce equivalent valuations over common variables.

For the case when p is a state without available outgoing transitions, the control will go back to the outmost source state as defined in the second transition rule for Stateflow chart. This is also achieved using the *out*! and *out*? between the source state and its decomposition inside in our translation.

- If p is a terminal junction, the execution process is part of the above case, and the proof is similar. If p is recursively a state vector, the case holds by structural induction.

From the above proof, starting from the states in the simulation set, for each one-step transition (including the transitions within one sample time step) of Stateflow, there is a corresponding transition of the translated TA such that the targets of the two transitions still satisfy the simulation relation. The proof of this theorem is completed.

10 Proof of Proposition 1

Given a Stateflow chart $\mathcal{D}$ and its transition system $T = (Q, \rightarrow, Q^0, V)$, we need to prove that, for every configuration C of T, there exists a successor C' such that $C \rightarrow C'$.

First of all, the initial set Q^0 is not empty. Furthermore, for each configuration C, it is a tuple of the form $(S_1, S_2, \cdots, S_n, v)$, then we need to prove that for each S_i, there is a possible transition path starting from a corresponding state, to reach a next state. Notice that, according to the semantics presented in Section 5.2, the disjunction of the transitions for each state or junction in T constitutes a total set. We show the case for a state S. The rules (1)-(5) define the different cases for execution of S. If there is an enabled transition from S to another state or junction, then the transitions in (1) or (3) is taken. Otherwise, the condition of transition (4) or (5) must be true and taken, depending on whether the during action is executed or not. For the special case that the during action is not explicitly defined, the transition (4) performs a time progress on the activation clock of current state (this is not considered as deadlock, and this time progress is explicitly preserved in the corresponding TA model). The transition (2) is enabled after an event occurs. The case for junction is different as defined in transition (6). But the fact also holds for it under the condition that no terminal junction exists. From the fact that the disjunction of transitions constitutes a total set, we can conclude that the transition relation $\rightarrow$ is total directly.